Multimodal visual understand application

Multimodal visual understand application

1. Description of the function
   1.1 The run flow
   1.2 The run conditions
2. Configuring the large model backend
   2.1 The online large model: Tongyi Qianwen
   2.2 The offline large model: Ollama
3. Text interaction
   3.1 Starting the program
   3.2 Making a visual request
4. Voice interaction
   4.1 Starting and testing
5. Acceptance and troubleshooting
   5.1 The acceptance steps
   5.2 Frequently asked questions

> **Hardware requirement: a Jetson with a 4GB configuration cannot run the cases of this tutorial; please use a Jetson with 8GB of memory or more.**

1. Description of the function

The visual understanding function takes the current image through the camera, then sends the image and the prompt to a multimodal large model that supports image input, and finally returns the analysis result as text or as speech. The course program completes the photo taking and the image analysis through `ToolsManager.seewhat()`, and the image inference is called by `model_interface.infer_with_image()` on the platform configured by `llm_platform`.

This article puts the online/offline large model and the text/voice interaction into the same set of operations. The differences between the four combinations are as follows:

Large model	Text interaction	Voice interaction
Online (for example Tongyi Qianwen)	`text_chat_mode:=true`, use the `text_chat` input	Start the voice chain; `llm_platform:'tongyi'`, the ASR/TTS can each choose online or offline
Offline (Ollama)	`text_chat_mode:=true`, use the `text_chat` input	Start the voice chain; `llm_platform:'ollama'`, the ASR/TTS can each choose online or offline

Online/offline describes the **large model backend**; the ASR (speech to text) and the TTS (text to speech) of the voice interaction are configured independently. The text interaction uses neither the microphone nor the speaker.

1.1 The run flow

1. `seewhat` asks the camera to take a frame.
2. The program hands the picture and the scene description prompt to `infer_with_image()`.
3. The model returns the text of the image analysis.
4. The text mode displays the result in the terminal; the voice mode hands the result to the TTS to announce it.

> Note: `seewhat` is a tool call. Whether it can be triggered automatically by a natural language question in the text mode depends on the current model and on the tool call logic of the course program; supporting image input does not mean that the model also supports tool calls. In the voice mode, the deployed version of the course can trigger the photo analysis directly by recognising the visual keywords in the text; if the source version you use does not have that logic, the model has to initiate the tool call correctly as well. When testing, check whether `Executing seewhat()` appears in the log; do not judge that the camera chain works from the text answer of the model alone.

1.2 The run conditions

- The `largemodel` workspace, the camera and the voice interaction environment have been configured as described in the previous tutorials.
- The camera is connected and the current user has access to it; the model has to be able to receive image input.
- The online large model needs an available network, valid platform credentials and a model that supports image input. The offline large model needs the Ollama service running, a model that supports image input to be downloaded, and enough free memory.
- The voice mode additionally needs a working microphone and speaker, the ASR/TTS configuration and the corresponding service or local model.

---

2. Configuring the large model backend

The configuration files are located at:

```
~/yahboom_ws/src/largemodel/config/yahboom.yaml
~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

After every backend switch, save the configuration and restart `largemodel`. Do not configure `llm_platform` with several values at the same time.

2.1 The online large model: Tongyi Qianwen

Tongyi Qianwen is used as an example here. First apply for it as described in the online large model basic tutorial and confirm that the account has available quota, and then open the model interface configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/large_model_interface.yaml
```

Find the Tongyi Qianwen configuration item, fill in your own API Key and select a vision model that supports image input. Do not keep the example placeholder as it is:

```
qianwen_api_key: "Replace it with your own API-Key"
qianwen_model: "qwen-vl-max-latest"
```

After saving the model interface configuration, open the main configuration file:

```
vim ~/yahboom_ws/src/largemodel/config/yahboom.yaml
```

Set `llm_platform` to `tongyi` in the existing `model_service.ros__parameters` parameter block. If the parameter already exists, simply modify the original value and do not add a duplicate YAML key with the same name:

```
model_service:
  ros__parameters:
    llm_platform: 'tongyi'
```

After saving both configuration files, restart `largemodel` so that the settings take effect. At runtime the Jetson has to be able to reach the online service, and the selected model must support image input. Never publish a real API Key to a public code repository or in a screenshot; for the names, the credentials and the vision model configuration of the other online platforms, follow the tutorial of the corresponding platform.

2.2 The offline large model: Ollama

Make sure the Ollama service is running and that a model supporting image input has been downloaded. Edit `large_model_interface.yaml`:

```
ollama_host: "http://localhost:11434"
ollama_model: "qwen3-vl:2b"  # Example; replace it with the vision model you have downloaded
```

Select Ollama in `yahboom.yaml`:

```
model_service:
  ros__parameters:
    llm_platform: 'ollama'
```

Check the Ollama models and the service of the machine:

```
ollama list
systemctl status ollama --no-pager
```

The model name has to match the name shown by `ollama list`. Different models support the image input and the tool calls differently; if the photo taking tool is not triggered in the text mode, check the tool call capability of the model and the program log first. If the voice chain is configured with the direct visual keyword path, you can verify that branch with an explicit visual request.

> The inference speed of the offline large model depends on the computing power of the Jetson mainboard and is usually slower than the online large model; the actual speed also depends on the size of the model, the image resolution and the network conditions.

3. Text interaction

The text interaction uses the `text_chat` node to enter the question and does not need the ASR/TTS to be started. First select the online or the offline large model backend as described in section 2.

3.1 Starting the program

Open the first terminal, enter the terminal in which the ROS 2 Jazzy and the workspace environment have been configured, and then start it:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=true
```

Open the second terminal, load the same ROS 2 and the workspace environment there as well, and then run:

```
ros2 run text_chat text_chat
```

3.2 Making a visual request

Enter a clear visual request in the `text_chat` terminal, for example:

```
Please call seewhat to take a photo of the current view and tell me what is in it.
```

Watch the `largemodel` terminal: if the log shows `Executing seewhat()`, the program has entered the photo taking and image analysis flow; the scene description returned by the model should then appear. When the test is over, press `Ctrl+C` in each of the two terminals to close the text client and the main program.

If the model only answers with text and there is no `Executing seewhat()`, the tool was not triggered; this does not mean that the camera took the photo successfully. Try a model that supports tool calls, or switch to the voice mode to verify the visual keyword trigger logic of the course version.

---

4. Voice interaction

The voice interaction receives the request through the microphone, triggers `seewhat` once it has recognised a visual question, and then announces the analysis result through the TTS. First select the large model backend as described in section 2, and then check the voice options of `model_service.ros__parameters` in `yahboom.yaml`:

```yaml
model_service:
  ros__parameters:
    llm_platform: 'ollama'  # Change to 'tongyi' for the online large model
    use_oline_asr: False    # True: online ASR; False: offline ASR
    useolinetts: False      # True: online TTS; False: offline TTS
    regional_setting: "China"
```

The example above uses the offline ASR and the offline TTS. If you want the speech recognition and the speech synthesis to use the online services as well, set `use_oline_asr` and `useolinetts` to `True` respectively and configure the service credentials and the network as described in the ASR/TTS tutorials. Whichever voice service you choose, the LLM backend is decided separately by `llm_platform`. For example, an online LLM with an offline voice frontend and backend is set to `tongyi` with the two voice parameters set to `False`; an offline LLM with an online ASR/TTS is set to `ollama` with the two voice parameters set to `True`.

4.1 Starting and testing

Make sure the microphone, the speaker and the voice related services are ready. Open a terminal, load the ROS 2 and the workspace environment and start it:

```
ros2 launch largemodel largemodel_control.launch.py
```

After waking the device up, ask a clear visual question, for example "What do you see?" or "Please describe the current view". Check the recognised text and the `Executing seewhat()` log in the startup terminal; shortly afterwards you should hear the TTS announcing the analysis result. Press `Ctrl+C` to stop the program.

---

5. Acceptance and troubleshooting

5.1 The acceptance steps

It is recommended to verify the basic chains separately first and then test the complete combination:

1. Check the offline service with `ollama list` and `systemctl status ollama`; in the online mode confirm the API Key, the model name and that the network is available.
2. Confirm that the camera device is visible:

   ```
   ls /dev/video*
   ```

3. Make an explicit photo analysis request in the text mode or the voice mode and confirm that `Executing seewhat()` appears in the log.
4. Confirm that the image model returns content; in the voice mode also confirm that the speaker announces the result.

5.2 Frequently asked questions

Symptom	Direction to check
The online model fails the authentication or the request times out	Check `qianwen_api_key`, the model name, the network connection and the permissions of the platform account; confirm that the selected model supports image input.
The Ollama connection is refused or the model is not found	Check the status of the Ollama service and the spelling of `ollama_host` and `ollama_model`, and confirm that the model has been downloaded.

Symptom	Direction to check
A text answer arrives but there is no photo taking log	The model may not have called `seewhat`. Confirm the tool call capability of the model; in the text mode do not treat the natural language answer as proof that the camera chain is running.
`seewhat` reports that the camera cannot be opened or the photo fails	Check the camera connection, the `/dev/video*` device nodes, whether the device is busy and the access permission of the current user.
The ASR returns no recognition result	Check the microphone connection, the `use_oline_asr` choice, the ASR credentials or the offline model, and the voice device parameters.
There is a text result but no voice announcement	Check `useolinetts`, the TTS service/model, the speaker and the audio output configuration.
The offline inference is very slow and the program is killed by the system	Check the available memory, verify with a smaller vision model and the text mode alone first, and close the other programs that use a lot of resources.